

Department Model Class

Use the following Department model class in the **Models** folder. It follows C# property naming conventions and correctly defines the Employees collection navigation property.

Understanding the Employees Collection

The **ICollection<Employee>** property represents a list of employees that belong to a single department.

For example, one department can contain many employee records:

IT Department

- John
- David
- Smith

HR Department

- Mary
- Kevin

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;

namespace EmployeeDepartmentApi.Models
{
    public class Department
    {
        public int DepartmentID { get; set; }
        public string DepartmentName { get; set; } = string.Empty;
        public ICollection<Employee> Employees { get; set; } = new List<Employee>();
    }
}
```

Understanding Get and Set Accessors

get and **set** are property accessors used to read and update the value of a property in C#.

Auto-Implemented Property

The following property lets you store and retrieve an employee's first name without writing a separate backing field manually.

```
public class Employee
{
    public string FirstName { get; set; }
}
```

Equivalent Long Form

The auto-implemented property above is shorthand for a full property that uses a private backing field.

```
private string _firstName;

public string FirstName
{
    get
    {
        return _firstName;
    }

    set
    {
        _firstName = value;
    }
}
```

Employee Model Class

Use the following Employee model class in the **Models** folder. It follows C# property naming conventions and correctly defines the Department navigation property.

Understanding the Department Relationship

The Employee model uses two properties to connect each employee to a department:

- **Foreign Key:** public int DepartmentID { get; set; } stores the ID of the department associated with the employee.
- **Navigation Property:** public Department Department { get; set; } allows the application to access the related Department object directly.

How It Is Used

Because of the navigation property, you can access a department value through an employee object.

```
employee.Department.DepartmentName
```

Example

```
Employee emp = new Employee();  
  
emp.Department.DepartmentName;
```

Output: IT

```
using System;  
using System.Collections.Generic;  
using System.Linq;  
using System.Web;  
  
namespace EmployeeDepartmentApi.Models  
{  
    public class Employee  
    {  
        public int EmployeeID { get; set; }  
        public string FirstName { get; set; }  
        public string LastName { get; set; }  
        public string Email { get; set; }  
        public int DepartmentID { get; set; }  
        public Department Department { get; set; }  
    }  
}
```

AppDbContext Class

Use the following **AppDbContext** class in the **Data** folder. It connects the Employee and Department models to Entity Framework Core and defines the database tables through DbSet properties.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using EmployeeDepartmentApi.Models;
using Microsoft.EntityFrameworkCore;

namespace EmployeeDepartmentApi.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)
            : base(options)
        {
        }

        public DbSet<Employee> Employees { get; set; }
        public DbSet<Department> Departments { get; set; }
    }
}
```

Understanding AppDbContext

AppDbContext is the database manager class for the application. It connects the application models to Entity Framework Core and represents the database tables used by the project.

Required Imports

The following imports are needed so the context class can access the model classes and Entity Framework Core features:

- **EmployeeDepartmentApi.Models** gives access to the Employee and Department classes.
- **Microsoft.EntityFrameworkCore** gives access to DbContext, DbSet, and other EF Core features.

```
using EmployeeDepartmentApi.Models;  
using Microsoft.EntityFrameworkCore;
```

Namespace and Class Definition

The namespace places the file in the Data layer of the project, while the class definition makes AppDbContext inherit from EF Core's DbContext base class.

```
namespace EmployeeDepartmentApi.Data  
  
public class AppDbContext : DbContext
```

Constructor and Database Options

The constructor receives database configuration settings from **Program.cs**. For example, a SQL Server connection string is passed into the context through the options parameter.

```
public AppDbContext(DbContextOptions<AppDbContext> options)  
    : base(options)  
{  
}  
}
```

DbSet Properties

Each **DbSet** property represents a table in the database. These properties allow the application to query, add, update, and delete records.

- **DbSet<Employee> Employees** represents the Employees table.
- **DbSet<Department> Departments** represents the Departments table.

Example Usage

After defining the DbSet properties, you can use the context object to work with employee records.

```
_context.Employees.ToListAsync();  
_context.Employees.Add(employee);
```

Summary

- **AppDbContext** is the database connection class.
- **DbContext** is the EF Core base class.
- **DbSet<Employee>** represents the Employees table.
- **DbSet<Department>** represents the Departments table.